

Transport Layer Security Cheat Sheet

Introduction

This cheat sheet provides guidance on implementing transport layer protection for applications using Transport Layer Security (TLS). It primarily focuses on how to use TLS to protect clients connecting to a web application over HTTPS, though much of this guidance is also applicable to other uses of TLS. When correctly implemented, TLS can provide several security benefits:

- **Confidentiality:** Provides protection against attackers reading the contents of the traffic.
- **Integrity:** Provides protection against traffic modification, such as an attacker replaying requests against the server.
- **Authentication:** Enables the client to confirm they are connected to the legitimate server. Note that the identity of the client is not verified unless [client certificates](#) are employed.

SSL vs TLS

Secure Socket Layer (SSL) was the original protocol that was used to provide encryption for HTTP traffic, in the form of HTTPS. There were two publicly released versions of SSL - versions 2 and 3. Both of these have serious cryptographic weaknesses and should no longer be used.

For [various reasons](#) the next version of the protocol (effectively SSL 3.1) was named Transport Layer Security (TLS) version 1.0. Subsequently TLS versions 1.1, 1.2 and 1.3 have been released.

The terms "SSL", "SSL/TLS" and "TLS" are frequently used interchangeably, and in many cases "SSL" is used when referring to the more modern TLS protocol. This cheat sheet will use the term "TLS" except where referring to the legacy protocols.

Server Configuration

Only Support Strong Protocols

Web applications must default to **TLS 1.3** and may support TLS 1.2 for compatibility. **TLS 1.0 and TLS 1.1 are formally deprecated by RFC 8996 (March 2021) and must be disabled.** They are also forbidden by [PCI DSS](#), disallowed by NIST SP 800-52 Rev. 2, and removed from all mainstream browsers. SSLv2 and SSLv3 must always be disabled.

If interoperability with end-of-life clients is a hard business requirement, isolate them on a dedicated endpoint with no access to sensitive data — do not weaken the primary endpoint. The ["TLS_FALLBACK_SCSV" extension](#) should be enabled to prevent protocol downgrade attacks.

Only Support Strong Ciphers

There are a large number of different ciphers (or cipher suites) that are supported by TLS, that provide varying levels of security.

For TLS 1.3, use the standard AEAD cipher suites (AES-GCM or ChaCha20-Poly1305).

If TLS 1.2 is still required, prefer AEAD-based suites there as well and avoid CBC-mode ciphers. At a minimum, the following types of [ciphersuites](#) should always be disabled:

- Null ciphers;
- Anonymous ciphers (`TLS_*_anon_*`);
- EXPORT ciphers (`TLS_*_EXPORT_*`);
- RSA transport (`TLS_RSA_*`) and ephemeral/static Diffie-Hellman key agreement (`TLS_DH_*` , `TLS_ECDH_*`) which do not provide forward secrecy.

The Mozilla Foundation provides an [easy-to-use secure configuration generator](#) for web, database and mail servers. This tool allows site administrators to select the software they are using and receive a configuration file that is optimized to balance security and compatibility for a wide variety of browser versions and server software.

Set the appropriate Diffie-Hellman groups

The practice of earlier than TLS 1.3 protocol versions of Diffie-Hellman parameter generation for use by the ephemeral Diffie-Hellman key exchange (signified by the "DHE" or "EDH" strings in the cipher suite name) had practical issues. For example, the client had no say in the selection of server parameters, meaning it could only unconditionally accept or drop, and the random parameter generation often resulted to denial of service attacks (CVE-2022-40735, CVE-2002-20001).

The `supported_groups` extension is used to negotiate which Diffie-Hellman group are supported:

- the available Finite Field Diffie-Hellman groups are `ffdhe2048` , `ffdhe3072` , `ffdhe4096` , `ffdhe6144` , `ffdhe8192` as specified in [RFC7919](#);
- Elliptic Curve Diffie-Hellman groups include `x25519` , `prime256v1` , `x448` , `secp384r1` ;
- for post-quantum cryptography, `X25519MLKEM768` it currently used (though it is not a group, proper).

For OpenSSL, the list of enabled groups can be configured ins in `openssl.cnf` . For example

```
openssl_conf = openssl_init  
[openssl_init]  
ssl_conf = ssl_module  
[ssl_module]  
system_default = tls_system_default  
[tls_system_default]  
Groups = X25519MLKEM768:x25519:prime256v1:x448:ffdhe2048:ffdhe3072
```

An Apache configuration would look like

```
SSLOpenSSLConfCmd Curves X25519MLKEM768:X25519:prime256v1:secp384r1
```

The same group on NGINX would look like the following

```
ssl_ecdh_curve X25519MLKEM768:X25519:prime256v1:secp384r1;
```

For TLS 1.2 or earlier versions it is recommended not to set Diffie-Hellman parameters.

Disable Compression

TLS compression should be disabled in order to protect against a vulnerability (nicknamed [CRIME](#)) which could potentially allow sensitive information such as session cookies to be recovered by an attacker.

Patch Cryptographic Libraries

As well as the vulnerabilities in the SSL and TLS protocols, there have also been a large number of historic vulnerability in SSL and TLS libraries, with [Heartbleed](#) being the most well known. As such, it is important to ensure that these libraries are kept up to date with the latest security patches.

Test the Server Configuration

Once the server has been hardened, the configuration should be tested. The [OWASP Testing Guide chapter on SSL/TLS Testing](#) contains further information on testing.

There are a number of online tools that can be used to quickly validate the configuration of a server, including:

- [SSL Labs Server Test](#)
- [CryptCheck](#)
- [Hardenize](#)

- [ImmuniWeb](#)
- [Observatory by Mozilla](#)
- [Scanigma](#)
- [Stellastra](#)
- [OWASP PurpleTeam](#) `cloud`

Additionally, there are a number of offline tools that can be used:

- [O-Saft - OWASP SSL advanced forensic tool](#)
- [CipherScan](#)
- [CryptoLyzer](#)
- [SSLScan - Fast SSL Scanner](#)
- [SSLyze](#)
- [testssl.sh - Testing any TLS/SSL encryption](#)
- [tls-scan](#)
- [OWASP PurpleTeam](#) `local`

Certificates

Use Strong Keys and Protect Them

The private key used to generate the cipher key must be sufficiently strong for the anticipated lifetime of the private key and corresponding certificate. The current best practice is to select a key size of at least 2048 bits when using RSA keys. Additional information on key lifetimes and comparable key strengths can be found [here](#) and in [NIST SP 800-57](#).

The private key should also be protected from unauthorized access using filesystem permissions and other technical and administrative controls.

Use Strong Cryptographic Hashing Algorithms

Certificates should use SHA-256 for the hashing algorithm, rather than the older MD5 and SHA-1 algorithms. These have a number of cryptographic weaknesses, and are not trusted by modern browsers.

Use Correct Domain Names

The domain name (or subject) of the certificate must match the fully qualified name of the server that presents the certificate. Historically this was stored in the `commonName` (CN) attribute of the certificate. However, modern versions of Chrome ignore the CN attribute, and

require that the FQDN is in the `subjectAlternativeName` (SAN) attribute. For compatibility reasons, certificates should have the primary FQDN in the CN, and the full list of FQDNs in the SAN.

Additionally, when creating the certificate, the following should be taken into account:

- Consider whether the "www" subdomain should also be included.
- Do not include non-qualified hostnames.
- Do not include IP addresses.
- Do not include internal domain names on externally facing certificates.
 - If a server is accessible using both internal and external FQDNs, configure it with multiple certificates.

Carefully Consider the use of Wildcard Certificates

Wildcard certificates can be convenient, however they violate [the principle of least privilege](#), as a single certificate is valid for all subdomains of a domain (such as *.example.org). Where multiple systems are sharing a wildcard certificate, the likelihood that the private key for the certificate is compromised increases, as the key may be present on multiple systems. Additionally, the value of this key is significantly increased, making it a more attractive target for attackers.

The issues around the use of wildcard certificates are complicated, and there are [various other discussions](#) of them online.

When risk assessing the use of wildcard certificates, the following areas should be considered:

- Only use wildcard certificates where there is a genuine need, rather than for convenience.
 - Consider the use of the [ACME](#) to allow systems to automatically request and update their own certificates instead.
- Never use a wildcard certificates for systems at different trust levels.
 - Two VPN gateways could use a shared wildcard certificate.
 - Multiple instances of a web application could share a certificate.
 - A VPN gateway and a public web server **should not** share a wildcard certificate.
 - A public web server and an internal server **should not** share a wildcard certificate.
- Consider the use of a reverse proxy server which performs TLS termination, so that the wildcard private key is only present on one system.
- A list of all systems sharing a certificate should be maintained to allow them all to be updated if the certificate expires or is compromised.

- Limit the scope of a wildcard certificate by issuing it for a subdomain (such as `*.foo.example.org`), or a for a separate domain.

Use an Appropriate Certification Authority for the Application's User Base

In order to be trusted by users, certificates must be signed by a trusted certificate authority (CA). For Internet facing applications, this should be one of the CAs which are well-known and automatically trusted by operating systems and browsers.

The [LetsEncrypt](#) CA provides free domain validated SSL certificates, which are trusted by all major browsers. As such, consider whether there are any benefits to purchasing a certificate from a CA.

For internal applications, an internal CA can be used. This means that the FQDN of the certificate will not be exposed (either to an external CA, or publicly in certificate transparency lists). However, the certificate will only be trusted by users who have imported and trusted the internal CA certificate that was used to sign them.

Use CAA Records to Restrict Which CAs can Issue Certificates

Certification Authority Authorization (CAA) DNS records can be used to define which CAs are permitted to issue certificates for a domain. The records contains a list of CAs, and any CA who is not included in that list should refuse to issue a certificate for the domain. This can help to prevent an attacker from obtaining unauthorized certificates for a domain through a less-reputable CA. Where it is applied to all subdomains, it can also be useful from an administrative perspective by limiting which CAs administrators or developers are able to use, and by preventing them from obtaining unauthorized wildcard certificates.

Consider the Certificate's Validation Type

Certificates come in different types of validation. Validation is the process the Certificate Authority (CA) uses to make sure you are allowed to have the certificate. This is authorization. The [CA/Browser Forum](#) is an organization made of CA and browser vendors, as well as others with an interest in web security. They set the rules which CAs must follow based on the validation type. The base validation is called Domain Validated (DV). All publicly issued certificates must be domain validated. This process involves practical proof of control of the name or endpoint requested in the certificate. This usually involves a challenge and response in DNS, to an official email address, or to the endpoint that will get the certificate.

Organization Validated (OV) certificates include the requestor's organization information in the certificates subject. E.g. C = GB, ST = Manchester, **O = Sectigo Limited**, CN = sectigo.com. The process to acquire an OV certificate requires official contact with the requesting company via a method that proves to the CA that they are truly talking to the right company.

Extended validation (EV) certificates provide an even higher level of verification as well as all the DV and OV verifications. This can effectively be viewed as the difference between "This site is really run by Example Company Inc." vs "This domain is really example.org". [Latest Extended Validation Guidelines](#)

Historically these displayed differently in the browser, often showing the company name or a green icon or background in the address bar. However, as of 2019 no major browser shows EV status like this as they do not believe that EV certificates provide any additional protection. ([Chromium](#) Covering Chrome, Edge, Brave, and Opera. [Firefox Safari](#))

As all browsers and TLS stacks are unaware of the difference between DV, OV, and EV certificates, they are effectively the same in terms of security. An attacker only needs to reach the level of practical control of the domain to get a rogue certificate. The extra work for an attacker to get an OV or EV certificate in no way increases the scope of an incident. In fact, those actions would likely mean detection. The additional pain in getting OV and EV certificates may create an availability risk and their use should be reviewed with this in mind.

Application

Use TLS For All Pages

TLS should be used for all pages, not just those that are considered sensitive such as the login page. If there are any pages that do not enforce the use of TLS, these could give an attacker an opportunity to sniff sensitive information such as session tokens, or to inject malicious JavaScript into the responses to carry out other attacks against the user.

For public facing applications, it may be appropriate to have the web server listening for unencrypted HTTP connections on port 80, and then immediately redirecting them with a permanent redirect (HTTP 301) in order to provide a better experience to users who manually type in the domain name. This should then be supported with the [HTTP Strict Transport Security \(HSTS\)](#) header to prevent them accessing the site over HTTP in the future.

API-only endpoints should disable HTTP altogether and only support encrypted connections. When that is not possible, API endpoints should fail requests made over unencrypted HTTP connections instead of redirecting them.

Do Not Mix TLS and Non-TLS Content

A page that is available over TLS should not include any resources (such as JavaScript or CSS) files which are loaded over unencrypted HTTP. These unencrypted resources could allow an attacker to sniff session cookies or inject malicious code into the page. Modern browsers will also block attempts to load active content over unencrypted HTTP into secure pages.

Use the "Secure" Cookie Flag

All cookies should be marked with the "Secure" attribute, which instructs the browser to only send them over encrypted HTTPS connections, in order to prevent them from being sniffed from an unencrypted HTTP connection. This is important even if the website does not listen on HTTP (port 80), as an attacker performing an active man in the middle attack could present a spoofed web server on port 80 to the user in order to steal their cookie.

Prevent Caching of Sensitive Data

Although TLS provides protection of data while it is in transit, it does not provide any protection for data once it has reached the requesting system. As such, this information may be stored in the cache of the user's browser, or by any intercepting proxies which are configured to perform TLS decryption.

Where sensitive data is returned in responses, HTTP headers should be used to instruct the browser and any proxy server not to cache the information, in order to prevent it being stored or returned to other users. For modern HTTP/1.1+ clients and intermediaries, a single header is sufficient:

```
Cache-Control: no-store
```

`no-store` is the strongest cache directive: it forbids both shared and private caches from storing any part of the response. The legacy combination `Cache-Control: no-cache, no-store, must-revalidate` plus `Pragma: no-cache` and `Expires: 0` is only required if you must support pre-HTTP/1.1 caches (effectively obsolete in 2024+) and adds no protection beyond `no-store` on a modern stack. Note that `Cache-Control` governs the HTTP cache; it does not control whether the browser stores cookies in its cookie jar — that is controlled by the cookie attributes (`Max-Age`, `Expires`, `Session`).

If you also need to clear data already cached on the client at sign-out, additionally send `Clear-Site-Data` (e.g. `Clear-Site-Data: "cache", "cookies", "storage"`).

Use HTTP Strict Transport Security

HTTP Strict Transport Security (HSTS) instructs the user's browser to always request the site over HTTPS, and also prevents the user from bypassing certificate warnings. See the [HTTP Strict Transport Security Cheat Sheet](#) for further information on implementing HSTS.

Client Certificates and Mutual TLS

In a typical TLS configuration, a certificate on the server allows the client to verify the server's identity and provides an encrypted connection between them. However, this approach has two

main weaknesses:

- The server lacks a mechanism to verify the client's identity.
- An attacker, obtaining a valid certificate for the domain, can intercept the connection. This interception is often used by businesses to inspect TLS traffic, by installing a trusted CA certificate on their client systems.

Client certificates, central to mutual TLS (mTLS), address these issues. In mTLS, both the client and server authenticate each other using TLS. The client proves their identity to the server with their own certificate. This not only enables strong authentication of the client but also prevents an intermediate party from decrypting TLS traffic, even if they have a trusted CA certificate on the client system.

Challenges and Considerations

Client certificates are rarely used in public systems due to several challenges:

- Issuing and managing client certificates involves significant administrative overhead.
- Non-technical users may find installing client certificates difficult.
- Organizations' TLS decryption practices can cause client certificate authentication, a key component of mTLS, to fail.

Despite these challenges, client certificates and mTLS should be considered for high-value applications or APIs, particularly where users are technically sophisticated or part of the same organization.

Public Key Pinning

Public key pinning can be used to provides assurance that the server's certificate is not only valid and trusted, but also that it matches the certificate expected for the server. This provides protection against an attacker who is able to obtain a valid certificate, either by exploiting a weakness in the validation process, compromising a trusted certificate authority, or having administrative access to the client.

Public key pinning was added to browsers in the HTTP Public Key Pinning (HPKP) standard. However, due to a number of issues, it has subsequently been deprecated and is no longer recommended or [supported by modern browsers](#).

However, public key pinning can still provide security benefits for mobile applications, thick clients and server-to-server communication. This is discussed in further detail in the [Pinning Cheat Sheet](#).

Related Articles

- [OWASP - Testing for Weak TLS](#)
- [OWASP - Application Security Verification Standard \(ASVS\) - Communication Security Verification Requirements \(V9\)](#)
- [Mozilla - Mozilla Recommended Configurations](#)
- [NIST - SP 800-52 Rev. 2 Guidelines for the Selection, Configuration, and Use of Transport Layer Security \(TLS\) Implementations](#)
- [NIST - NIST SP 800-57 Recommendation for Key Management, Revision 5](#)
- [NIST - SP 800-95 Guide to Secure Web Services](#)
- [IETF - RFC 5280 Internet X.509 Public Key Infrastructure Certificate and Certificate Revocation List \(CRL\) Profile](#)
- [IETF - RFC 2246 The Transport Layer Security \(TLS\) Protocol Version 1.0 \(JAN 1999\)](#)
- [IETF - RFC 4346 The Transport Layer Security \(TLS\) Protocol Version 1.1 \(APR 2006\)](#)
- [IETF - RFC 5246 The Transport Layer Security \(TLS\) Protocol Version 1.2 \(AUG 2008\)](#)